

Contents

guide_01_distserve: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving 1

- 0. 这篇论文到底解决什么问题 2
 - 1. 当时的技术语境 3
 - 2. 术语先打牢 3
 - 2.1 Prefill 3
 - 2.2 Decode 4
 - 2.3 TTFT, TPOT, SLO attainment 4
 - 2.4 Goodput 4
 - 3. 论文的核心洞察: prefill 和 decode 不该被迫同居 4
 - 3.1 干扰来自哪里 4
 - 3.2 Chunked prefill 为什么还不够 5
 - 3.3 资源和 parallelism 被绑死 5
 - 4. DistServe 的系统设计 6
 - 4.1 总体架构 6
 - 4.2 KV cache 到底传什么 6
 - 4.3 高 node-affinity 集群的 placement 7
 - 4.4 低 node-affinity 集群的 placement 7
 - 4.5 Online scheduling 8
 - 5. 数学和排队模型怎么读 9
 - 5.1 M/D/1 直觉 9
 - 5.2 Inter-op 和 intra-op 的 tradeoff 9
 - 5.3 Decode 阶段的数学直觉 10
 - 6. 论文实验证据链 10
 - 6.1 实验平台和 workload 10
 - 6.2 Baselines 是否够强 11
 - 6.3 主结果 11
 - 6.4 传输开销是否真的不大 12
 - 6.5 Simulator 和 ablation 12
 - 6.6 99% SLO attainment 的 appendix 13
 - 7. 新手最容易误读的地方 13
 - 8. 局限性和边界 13
 - 9. 和本仓库代码怎么连起来 14
 - 9.1 原有最小实验 14
 - 9.2 新增 DistServe toy simulator 14
 - 9.3 建议你亲手做的 4 个实验 15
 - 10. 张量级别流程图 16
 - 11. 用 AI agent 正确学习这篇论文 17
 - 12. 闭卷自测 18
 - 13. 一句话复述模板 18

guide_01_distserve: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

原论文: DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

本地原文 PDF: [learning/distributed-inference/paper/01_distserve.pdf](#)

作者: Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, Hao Zhang

机构: Peking University, StepFun, UC San Diego

版本: arXiv 2401.09670v3, 2024-06-06

类型: LLM serving system paper, OSDI 2024 时代的分离式推理架构论文

0. 这篇论文到底解决什么问题

一句话: DistServe 发现 LLM 在线服务不能只看总吞吐, 因为用户真实感知的是两个不同延迟指标: TTFT 和 TPOT。它把 prefill 和 decoding 拆到不同 GPU 资源池, 分别做 batching, parallelism 和 placement, 用 per-GPU goodput 作为优化目标, 在满足 SLO attainment 的前提下降低每个请求的 GPU 成本。

如果你只记一个图, 记这个:

传统 colocated serving:

```
incoming requests
  |
  v
same GPU workers
  |
  +-- prefill for new request
  +-- decode step for old requests
  +-- prefill for another new request
  +-- decode step for old requests
```

问题:

long prefill blocks decode steps, so TPOT gets worse.

decode steps share the same batch and resource plan, so TTFT also gets worse.

one pool has to satisfy two different latency targets at the same time.

DistServe 的图是:

```
incoming request
  |
  v
controller
  |
  v
prefill pool
  |
  |   KV cache transfer
  v
decode pool
  |
  v
stream output tokens
```

这不是为了让系统看起来更复杂, 而是为了让两个阶段各自按自己的目标扩容:

- prefill 目标: 降低 TTFT, 通常要处理整段 prompt, 对 compute 更敏感。
- decode 目标: 降低 TPOT, 每次只生成一个 token, 对 memory bandwidth, KV cache 和 batch size 更敏感。
- placement 目标: 拆开以后必须传 KV cache, 所以要把通信成本控制住。
- optimization 目标: 在 TTFT 和 TPOT 都达标的情况下, 最大化每张 GPU 的可服务请求率。

1. 当时的技术语境

在 DistServe 之前, LLM serving 已经有几个非常重要的工程方向:

- Orca / continuous batching: 把不同请求的 prefill 和 decode step 混在一起批处理, 提高 GPU 利用率。
- vLLM / PagedAttention: 重点解决 KV cache 显存碎片和动态分配, 让高并发推理更可控。
- DeepSpeed-MII / chunked prefill: 把长 prefill 切成小块, 让 decode step 可以插进来, 缓解长 prompt 对 decode 的阻塞。
- Tensor parallelism / pipeline parallelism: 用多 GPU 承载大模型, 或降低单次执行延迟。

这些工作很强, 但它们通常默认一个架构: prefill 和 decode 在同一组 GPU 上共存。这个默认设置在追求总 tokens/s 时很自然, 因为两阶段共享模型权重和 KV cache, colocate 看起来省显存, 也省通信。

DistServe 的切入点是: 在线 LLM 服务不是离线批处理。用户不是只在意 tokens/s, 而是在意:

- 第一口响应什么时候出来: TTFT, time to first token。
- 后续 token 是否稳定流出来: TPOT, time per output token。
- 有多少请求满足这两个延迟约束: SLO attainment。
- 每张 GPU 能承载多少达标请求: per-GPU goodput。

论文第一张图用一个 13B LLM 的 synthetic workload 做了很直观的铺垫: input length = 512, output length = 64, 单张 NVIDIA A100 80GB。colocated existing system 在 90% SLO attainment 下, 单 GPU 大约只能做到 1.6 rps。可是如果单独看 prefill-only, 可以到 5.6 rps; 单独看 decode-only, 可以到 10 rps。也就是说, 两个阶段分开时都还挺能跑, 混在一起反而互相拖后腿。

作者进一步指出: 如果理想地用 2 张 GPU 给 prefill, 1 张 GPU 给 decode, 整体可以达到 10 rps, 也就是 3.3 rps per GPU, 比 colocated 的 1.6 rps per GPU 高 2.1x。这个 toy observation 是整篇论文的出发点。

2. 术语先打牢

2.1 Prefill

prefill 是处理用户 prompt 的阶段。输入是已有 token 序列:

tokens: $[x_1, x_2, \dots, x_L]$

hidden states per layer: $[batch, prompt_len, hidden_dim]$

模型一次性处理整段 prompt, 生成第一个输出 token 的 logits, 同时为每一层产生后续 decode 要用的 KV cache。

直觉:

- prompt 很长时, prefill 一次计算量很大。
- 很多矩阵乘法可以并行处理整段序列。
- 对大 prompt, prefill 往往更接近 compute-bound。
- 用户感知上, prefill 决定 TTFT。

2.2 Decode

decode 是后续逐 token 生成阶段。每一步输入通常只有新生成的一个 token, 但注意力要读过去所有 tokens 的 KV cache:

```
new token hidden: [batch, 1, hidden_dim]
past KV per layer: [batch, past_len, kv_heads, head_dim]
next token logits: [batch, vocab_size]
```

直觉:

- 每一步只生成一个 token。
- 算术量比 prefill 小, 但要反复读取权重和 KV cache。
- 单个 decode step 容易 memory-bandwidth-bound。
- 需要足够大的 decode batch 才能提高 GPU 利用率。
- 用户感知上, decode 决定 TPOT。

2.3 TTFT, TPOT, SLO attainment

TTFT:

TTFT = time from request arrival to first generated token

TPOT:

TPOT = average time per output token after the first token

论文脚注也强调: end-to-end latency 大致可以理解为:

```
request_latency = TTFT + TPOT * number_of_generated_tokens
```

SLO attainment:

```
request_passes = (TTFT <= SLO_TTFT) and (TPOT <= SLO_TPOT)
attainment = passed_requests / total_requests
```

DistServe 常用 90% attainment 做主要阈值, 也在 appendix 里看 99% 更严格目标。

2.4 Goodput

传统 throughput 往往是:

```
throughput = generated_tokens / second
```

DistServe 关心的是:

```
goodput = max request_rate that still meets the SLO attainment target
per_gpu_goodput = goodput / number_of_GPUs
```

这个定义很重要。一个系统即使 tokens/s 很高, 如果大量请求违反 TTFT 或 TPOT, 对在线服务来说也不是好系统。DistServe 要优化的是“花同样 GPU 钱时, 能服务多少合格请求”。

3. 论文的核心洞察: prefill 和 decode 不该被迫同居

3.1 干扰来自哪里

prefill 和 decode colocate 后, continuous batching 会把新请求的 prefill 和老请求的 decode step 放进同一批执行。这样能提高 GPU 利用率, 但带来两个延迟问题:

```

long prefill job arrives
    |
    v
decode steps wait for the long prefill batch
    |
    v
TPOT of active users increases

```

以及:

```

many decode jobs active
    |
    v
new request prefill shares GPU slots
    |
    v
TTFT of new user increases

```

论文 Figure 2 做的是—个非常有教学价值的 profiling: 在 13B 模型上, 比较 decoding-only batch 和 decoding batch 里额外加一个 prefill job 的执行时间。结论是, 加入一个 prefill job 会显著拖慢 decoding batch; prompt 越长, decode slowdown 越明显。同时, decode job 也会让 prefill 变慢。

这说明问题不只是“batch size 没调好”。只要两阶段在同一组 GPU 上抢同一份执行时间, 就存在结构性干扰。

3.2 Chunked prefill 为什么还不够

chunked prefill 的想法是把长 prompt 切成多个 chunk, 每个 chunk 和一些 decode job 混合执行。它能缓解单个长 prefill 一次性霸占 GPU 的问题, 但论文指出它仍然有 tradeoff:

- 它没有消除 prefill 和 decode 的资源竞争, 只是把长阻塞切碎。
- 如果想保护 TPOT, prefill 的 TTFT 会变差。
- 如果想保护 TTFT, decode 的 TPOT 会变差。
- 对长上下文, chunked prefill 会反复读取历史 KV cache, 额外增加内存访问。

论文里给的直觉是: 如果 prefill 被切成 N 个 chunk, 后续 chunk 需要反复读之前 chunk 的 KV cache, 总读取量从 $O(N)$ 变成 $O(N^2)$ 级别的模式。这里不要死背复杂细节, 关键是明白: chunking 把一次大阻塞拆成多次小阻塞, 但没有改变两个阶段共享资源这个根因。

3.3 资源和 parallelism 被绑死

prefill 和 decode 对 parallelism 的偏好不同:

- prefill: prompt 长时更 compute-bound, 更可能受益于 intra-op parallelism 来降低一次 prefill 执行时间。
- decode: 单步更 memory-bound, 需要大 decode batch; 在大 batch 时可以用 model parallelism 扩展 batch 容量和吞吐。
- 不同应用的 TTFT 和 TPOT 权重不同, 所以最优 split 也不同。

如果 colocated, 一个模型实例只能用一套 parallelism plan 同时服务 prefill 和 decode。这就像让两种完全不同的工作负载共用同一个排班表: 谁的 SLO 更紧, 系统就被迫向谁倾斜, 另一个指标可能浪费资源或被拖垮。

DistServe 的主张是: 把阶段拆开后, prefill pool 和 decode pool 可以各自选择 GPU 数量, tensor parallelism, pipeline parallelism, batching 策略和 replica 数量。

4. DistServe 的系统设计

4.1 总体架构

DistServe 是一个 orchestration layer, 架在 LLM inference engine 之上。论文中的实现包括:

- placement algorithm module
- RESTful API frontend
- orchestration layer
- parallel execution engine

runtime 请求路径是:

```
client
|
v
OpenAI-compatible frontend
|
v
central controller
|
+--> prefill instance with shortest queue
      |
      | produces first token and KV cache
      v
      KV cache stays or transfers
      |
      v
      decode instance with light load
      |
      v
      stream tokens back
```

注意这个设计里, prefill instance 和 decode instance 都保存一份模型权重。代价是显存和部署复杂度上升; 收益是两阶段的运行节奏分开。

4.2 KV cache 到底传什么

Transformer decode 需要读过去 token 的 K 和 V。对于每个 layer, 每个 prompt token 都会产生 K/V 向量:

```
For each layer:
  K: [batch, prompt_len, kv_heads, head_dim]
  V: [batch, prompt_len, kv_heads, head_dim]
```

一个粗略的 KV 大小估算:

```
kv_bytes_per_token =
    2                # K and V
    * num_layers
    * num_kv_heads
```

```
* head_dim
* bytes_per_value
```

```
kv_payload_bytes = prompt_tokens * kv_bytes_per_token
transfer_ms = kv_payload_bytes * 8 / bandwidth_bits_per_second * 1000
```

论文给了一个很有冲击力的例子: OPT-66B 上, 单个 512-token request 的 KV cache 大约 1.13GB。如果 arrival rate 是 10 rps, 就需要每秒传 11.3GB, 大约 90Gbps 才能让传输开销不显眼。

所以 DistServe 不是说“拆开一定免费”。它的工程判断是:

- 现代大 GPU 节点内部 NVLINK 带宽很高。
- 跨节点 InfiniBand 在一些集群里也足够高, 但不是所有集群都高。
- placement algorithm 必须感知带宽, 不能随便把 prefill 和 decode 放远。

4.3 高 node-affinity 集群的 placement

如果跨节点网络足够快, KV cache transfer 的约束较弱。DistServe 的 Algorithm 1 做两层搜索:

Input:

```
model
cluster size
GPU memory capacity
workload distribution
target traffic rate
```

Step 1:

```
enumerate feasible parallel configs for prefill
simulate prefill goodput under TTFT SLO
choose best prefill config
```

Step 2:

```
enumerate feasible parallel configs for decode
simulate decode goodput under TPOT SLO
choose best decode config
```

Step 3:

```
replicate prefill and decode instances until target rate is met
```

这里的关键是 simulator。真实跑每个 placement 太慢, 所以 DistServe 根据 workload 分布, input/output length, arrival process, model profile 和 parallelism plan, 估算每个配置的 SLO attainment, 然后用 binary search 找最大可服务 rate。

论文说 Algorithm 1 复杂度是 $O(N \cdot M^2)$, N 是每个 instance 的 node limit, M 是每个 node 的 GPU 数。因为 M 通常是 8, 这个搜索空间是可控的。

4.4 低 node-affinity 集群的 placement

如果跨节点带宽有限, 直接让任意 prefill instance 给任意 decode instance 传 KV 就会很贵。Algorithm 2 的思想是: 利用 pipeline/inter-op parallelism 把模型层分成 stage, 然后把 prefill 和 decode 的相同 stage 放在同一个物理节点内, 让 KV cache transfer 走 NVLINK。

可以画成这样:

```

node 0:
  prefill stage 0 <-> decode stage 0
  transfer via NVLINK

node 1:
  prefill stage 1 <-> decode stage 1
  transfer via NVLINK

node 2:
  prefill stage 2 <-> decode stage 2
  transfer via NVLINK

```

这比高带宽场景多了约束: 同一个 stage 的 prefill 和 decode 要尽量 colocate 在同一 node 内。优点是控制 KV transfer 延迟; 缺点是搜索空间和 placement 约束变复杂, 可能错过一些理论上更高 goodput 的 placement。

4.5 Online scheduling

DistServe runtime 用了相对朴素但清晰的在线策略:

- incoming requests 到 central controller。
- prefill 分配到 shortest queue 的 prefill instance。
- decode 分配到 least loaded 的 decode instance。
- 基础策略是 FCFS。

然后加了几个系统细节:

减少 pipeline bubbles:

- prompt length 不均匀会让 pipeline stage 出现气泡。
- DistServe 用 token 数估计 batch 执行时间。
- 对 prefill, 先 profile 一个让 GPU 饱和的最短 prompt length L_m 。
- prompt 短于 L_m 时可以组合 batch, prompt 长于 L_m 时单独处理。

应对 burstiness:

- 如果 prefill 完成后主动 push 大量 KV cache 到 decode, decode GPU 可能显存爆。
- DistServe 用 pull 模式: decode instance 需要时再从 prefill instance 拉 KV cache。
- 这样 prefill GPU 的显存可以作为一个临时队列缓冲区。

周期性 replanning:

- workload 的平均 input length, output length, arrival rate 可能变。
- 如果变化明显, workload profiler 触发重新跑 placement algorithm。
- 论文强调搜索通常是秒级到分钟级, 远小于真实 workload 小时级变化。

故障和抢占:

- 论文没有把 preemption 和 fault tolerance 做成核心贡献。
- 作者承认 FCFS 会有 convoy effect, 长请求可能挡住短请求。
- 也承认 prefill 和 decode 的依赖关系可能让故障传播更复杂。

5. 数学和排队模型怎么读

这篇论文不是纯数学论文, 但它用排队论解释了为什么不同 parallelism 在不同 rate 下表现不同。新手读的时候不要被公式吓住, 先把每个变量翻译成系统含义。

5.1 M/D/1 直觉

论文先考虑 prefill-only instance, 假设:

- 请求到达是 Poisson process。
- 每个请求 prefill 执行时间固定为 D 。
- 请求率是 R 。
- 利用率条件是 $R * D < 1$ 。

平均 TTFT 写作:

$$\text{Avg_TTFT} = D + R * D^2 / (2 * (1 - R * D))$$

拆开看:

D
execution time

$R * D^2 / (2 * (1 - R * D))$
queueing delay

这个式子的直觉:

- 当 rate 低时, queueing delay 小, latency 主要看 execution time。
- 当 rate 接近 capacity 时, $1 - R*D$ 变小, queueing delay 会急剧上升。
- 所以一个 placement 不能只看单请求 latency, 还必须看请求率下的排队。

5.2 Inter-op 和 intra-op 的 tradeoff

对 prefill, 如果用 inter-op parallelism, 模型层被切到不同 stage。它可能提高吞吐, 但单请求要过 pipeline, 不一定最快。

论文用:

$$\text{Avg_TTFT_inter} = D_s + R * D_m^2 / (2 * (1 - R * D_m))$$

其中:

- D_s 是请求走完整个 pipeline 的 service time。
- D_m 是最慢 stage 的 service time。

对 intra-op parallelism, 假设 2-way parallelism 的 speedup 是 K , 且 $1 < K < 2$, 因为通信开销导致不能完美 2 倍加速。论文写出:

$$\text{Avg_TTFT_intra} = D/K + R * D^2 / (2 * K * (K - R * D))$$

这两类公式给出的判断是:

- rate 低时, execution time 是主导项, intra-op 降低单请求执行时间更有价值。
- rate 高时, queueing delay 是主导项, inter-op 或 replication 对吞吐和排队更有价值。
- SLO 越紧, 越需要降低单请求 execution time。
- speedup K 越差, intra-op 的收益越小。

DistServe 的 placement search 本质上就是把这些 tradeoff 自动化, 而不是让工程师手猜。

5.3 Decode 阶段的数学直觉

decode 的关键不是单步计算量大, 而是每一步都要读模型权重和 KV cache。单个 decode request 往往不能把 GPU 吃满, 所以 batch size 很重要。

可以这样理解:

`decode_step_time = read_weights + read_KV + attention_and_MLP_compute`

single request:

- low arithmetic intensity
- poor GPU utilization

large decode batch:

- more tokens per step
- better amortization of weight reads
- higher GPU utilization

但 colocated 系统里, decode batch 想变大又会和 prefill 的 TTFT 目标冲突。DistServe 拆开后, decode pool 可以积累更适合 decode 的 batch, prefill pool 则按 TTFT 目标服务新请求。

6. 论文实验证据链

6.1 实验平台和 workload

论文的真实系统部署在:

- 4 个节点。
- 每节点 8 张 NVIDIA SXM A100 80GB。
- 节点内 GPU 用 NVLINK。
- 跨节点带宽 25Gbps。
- 因为跨节点带宽有限, 大部分实验用 low node-affinity placement algorithm。

模型:

- OPT-13B, model size 26GB。
- OPT-66B, model size 132GB。
- OPT-175B, model size 350GB。

任务和 SLO:

Chatbot, OPT-13B:

- dataset: ShareGPT
- TTFT SLO: 0.25s
- TPOT SLO: 0.1s

Chatbot, OPT-66B:

- dataset: ShareGPT
- TTFT SLO: 2.5s
- TPOT SLO: 0.15s

Chatbot, OPT-175B:

```
dataset: ShareGPT
TTFT SL0: 4.0s
TPOT SL0: 0.2s
```

Code completion, OPT-66B:

```
dataset: HumanEval
TTFT SL0: 0.125s
TPOT SL0: 0.2s
```

Summarization, OPT-66B:

```
dataset: LongBench
TTFT SL0: 15s
TPOT SL0: 0.15s
```

输入输出长度分布也很重要:

- ShareGPT 平均 input 约 755.5 tokens, output 约 200.3 tokens。
- HumanEval 平均 input 约 171.3 tokens, output 约 98.2 tokens。
- LongBench 平均 input 约 1738.3 tokens, output 约 90.7 tokens。

这三个任务正好代表不同侧重点:

- Chatbot: TTFT 要快, TPOT 也不能太慢。
- Code completion: 作为实时 coding assistant, TTFT 特别紧。
- Summarization: input 很长, TTFT 可以宽松一些, 但 TPOT 要快。

6.2 Baselines 是否够强

论文比较了两个主要 baseline:

vLLM:

- 支持 continuous batching。
- 支持 PagedAttention。
- 广泛使用, 是很合理的 serving baseline。
- 但 prefill 和 decode colocate。

DeepSpeed-MII:

- 支持 chunked prefill。
- 能缓解长 prefill 的干扰。
- 但仍然没有完全拆开 prefill 和 decode。
- 在 OPT-175B 某些配置上有 kernel 和 OOM 限制, 所以不能完整服务所有 case。

这个 baseline 设置整体是有说服力的: 一个是现代通用 LLM serving baseline, 一个是对 prefill 干扰做过工程缓解的 baseline。

6.3 主结果

论文总述结论:

```
DistServe can serve up to 7.4x more requests
or support 12.6x tighter SL0,
while keeping more than 90% requests within latency constraints.
```

具体任务:

Chatbot on ShareGPT:

- DistServe 相比 vLLM 可承载 2.0x 到 4.6x 更高 request rate。
- 相比 DeepSpeed-MII 可承载约 1.6x 到 2.4x 更高 request rate。
- 对 OPT-175B, placement search 找到了非平凡配置: prefill 和 decode 使用不同 inter-op / intra-op 组合。这说明手工猜 placement 很难。

Code completion on HumanEval:

- DistServe 对 vLLM: 5.7x 更高 request rate, 1.4x 更紧 SLO。
- DistServe 对 DeepSpeed-MII: 1.6x 更高 request rate, 1.4x 更紧 SLO。
- 这里 TTFT 很紧, DistServe 通过消除 decode interference 并自动增加 prefill intra-op parallelism, 降低 prefill latency。

Summarization on LongBench:

- DistServe 对 vLLM: 4.3x 更高 request rate, 12.6x 更紧 SLO。
- DistServe 对 DeepSpeed-MII: 1.8x 更高 request rate, 2.6x 更紧 SLO。
- LongBench input 长, prefill 压力大, 但任务给了宽松 TTFT, 所以 TPOT 更关键。vLLM colocate 时, 长 prefill 会让 decode TPOT 明显恶化。

6.4 传输开销是否真的不大

这是 DistServe 必须证明的一点。拆开两阶段后, 如果 KV cache transfer 太贵, 整个方案就会失败。

论文 Figure 10 做 latency breakdown, 用 OPT-175B on ShareGPT, 因为大模型 KV cache transfer 更有压力。结果:

- 即使是 OPT-175B, KV cache transmission 在总 latency 中也小于 0.1%。
- 从 transmission time CDF 看, 超过 95% 请求的传输时间小于 30ms。
- 关键原因是 low node-affinity algorithm 把对应 stage 尽量放在同一个机器上, 使用节点内 NVLINK。

这条证据不是说所有集群传输都免费, 而是说: 在 placement 做对, 且节点内带宽足够的情况下, 传输不是主瓶颈。

6.5 Simulator 和 ablation

DistServe 的 placement search 依赖 simulator, 所以论文必须证明 simulator 不离谱。Table 2 比较了 simulator 和 real system 在不同 rate 下的 SLO attainment, 误差小于 2%。这给 placement algorithm 的可信度提供支撑。

Ablation 主要看两个创新:

- disaggregation 本身。
- placement searching algorithm。

论文构造了 vLLM++: 让 vLLM 枚举不同 parallelism strategy 并选择最好。结果 vLLM++ 和 vLLM 表现相同, 因为默认 intra-op=4 已经是最佳 per-GPU goodput。这说明在 colocated 架构里, 只调 parallelism 没法释放被 prefill-decode interference 锁住的性能。

DistServe-High 和 DistServe-Low 的比较说明:

- 高跨节点带宽假设下, placement 更自由, goodput 更高。
- 低跨节点带宽下, placement 受同 stage 同 node 约束, 但仍然优于 colocated baselines。

6.6 99% SLO attainment 的 appendix

论文 appendix 还把 SLO attainment target 从 90% 提到 99%。结论是 DistServe 仍然相对强:

- 相比 vLLM, 可持续约 3x 到 8x 更高 rate, 或 1.24x 到 6.67x 更紧 SLO。
- 相比 DeepSpeed-MII, 可达到约 1.32x 到 8x 更高 rate, 或 1.20x 到 1.58x 更紧 SLO。

这说明论文不是只在 90% 这个相对宽松指标上成立。

7. 新手最容易误读的地方

误读 1: “ DistServe 就是多用几张 GPU。 ”

不是。论文指标是 per-GPU goodput。如果只是堆 GPU, per-GPU 指标不一定上升。DistServe 的贡献是用更合适的 split 和 placement, 让同样 GPU 数服务更多达标请求。

误读 2: “ prefill compute-bound, decode memory-bound, 所以永远 prefill 多给 GPU。 ”

不一定。资源分配取决于 workload 和 SLO。长 prompt 且 TTFT 紧时, prefill 可能需要更多 GPU; TPOT 紧或 output 长时, decode 可能需要更多资源。你在本仓库新增的 `distserve_original_minimal.py` 里可以看到 toy search 会随 SLO 变化选择不同 split。

误读 3: “ KV transfer 小于 0.1%, 所以可以忽略网络。 ”

不能这么读。传输小是 placement 做对后的结果, 而且依赖节点内 NVLINK 或足够好的跨节点带宽。论文第 7 节明确讨论了资源受限或网络受限场景。

误读 4: “ chunked prefill 已经解决了问题。 ”

chunked prefill 缓解阻塞, 但没有消除两个阶段共享 GPU 的结构性竞争。DistServe 的证据链正是在证明 disaggregation 带来的额外自由度是关键。

误读 5: “ SLO Scale 越小越好, 看主图就行。 ”

要看 rate 和 SLO 两条曲线。一个系统可能在低 rate 下满足很紧 SLO, 但 rate 稍高就崩。goodput 看的就是“ 在 SLO target 下能承载的最高 request rate ”。

8. 局限性和边界

论文自己在 Discussion 里讲得比较诚实。

吞吐优先的离线场景:

- 如果应用不在乎 latency, 只想最大化 total tokens/s, DistServe 不一定是最佳选择。
- chunked prefill with piggyback 这类系统可能更适合填满每个 iteration, 追求 GPU utilization。

资源受限场景:

- 如果只有少量 GPU, 甚至单 GPU, disaggregation 的设计空间很小。
- prefill 和 decode 各存一份权重可能无法承受。
- 简单 colocated serving 可能更实用。

超长上下文:

- 长上下文会增大 KV cache transfer。
- 但 prefill compute 也会随上下文变得更重, prefill/decode 差异更明显。
- 论文认为 disaggregation 对长上下文仍然有潜力, 但需要继续验证。

故障和抢占:

- DistServe 没有完整实现 advanced preemption 和 fault tolerance。
- FCFS 会有 convoy effect。
- prefill 和 decode 相互依赖, 一个 decode instance 故障可能影响多个 prefill instance 对应的请求。

多租户和公平性:

- 论文主要优化单模型或特定 workload 的 goodput。
- 多模型、多租户、优先级、公平性、成本隔离都不是主贡献。

9. 和本仓库代码怎么连起来

本专题对应:

- lecture: learning/distributed-inference/lectures/08-disaggregated.md
- KV transfer: learning/distributed-inference/src/kv_transfer_mock.py
- colocated vs disagg mock: learning/distributed-inference/src/disaggregated_mock.py
- capstone 3-config benchmark: learning/distributed-inference/src/capstone_disagg.py
- 本 guide 新增论文形状 toy simulator: learning/distributed-inference/src/distserve_original_minimal.py
- 新增测试: learning/distributed-inference/src/tests/test_distserve_original_minimal.py

9.1 原有最小实验

disaggregated_mock.py 做了三种配置:

```
colocate
disagg-near
disagg-remote
```

它强调的是:

- near disagg 可以让 prefill 和 decode 重叠。
- remote disagg 会受到跨节点带宽影响。
- 长 prompt 时, disagg-near 对 throughput 更友好。

这是 lecture 级别的直觉实验。

9.2 新增 DistServe toy simulator

distserve_original_minimal.py 更贴近论文:

```
from distserve_original_minimal import (
    Request,
    SLO,
    Placement,
    colocated_placement,
    goodput_at_rate,
    search_gpu_split,
)

requests = [
    Request(prompt_tokens=1024, output_tokens=96),
    Request(prompt_tokens=1536, output_tokens=96),
    Request(prompt_tokens=2048, output_tokens=128),
```

```

]

slo = SLO(ttft_ms=900, tpot_ms=13)

colocated = colocated_placement(total_gpus=3)
distserve = Placement(
    name="distserve-p1-d2",
    colocated=False,
    prefill_gpus=1,
    decode_gpus=2,
)

print(goodput_at_rate(requests, colocated, slo, request_rate_rps=1.1))
print(goodput_at_rate(requests, distserve, slo, request_rate_rps=1.1))

```

这个 toy simulator 把论文变量映射成可测函数:

```

Request:
    prompt_tokens
    output_tokens

SLO:
    ttft_ms
    tpot_ms

Placement:
    colocated or disaggregated
    prefill_gpus
    decode_gpus
    bandwidth_gbps

Metrics:
    TTFT
    TPOT
    attainment
    goodput_rps
    per_gpu_goodput_rps

```

你不该把这个 simulator 当成论文真实系统的复现。它的价值是强迫你理解变量之间的因果:

- prompt 越长, prefill 和 KV transfer 越重。
- output 越长, decode queue 越重。
- TPOT SLO 越紧, decode resource 越关键。
- TTFT SLO 越紧, prefill resource 越关键。
- bandwidth 越低, disaggregation 的 TTFT 成本越高。
- search 结果不是固定的, 而是由 workload 和 SLO 共同决定。

9.3 建议你亲手做的 4 个实验

实验 1: 改 SLO 看 split 变化。

case A:
 TTFT tight, TPOT loose
 expect: more prefill GPUs

case B:
 TTFT loose, TPOT tight
 expect: more decode GPUs

实验 2: 改 bandwidth。

near:
 bandwidth_gbps = 900

far:
 bandwidth_gbps = 25

observe:
 TTFT changes because KV transfer is on the path to first decode.

实验 3: 改 prompt length。

prompt_tokens from 512 to 4096
observe:
 prefill time grows
 KV transfer grows
 TTFT SLO becomes harder

实验 4: 改 output length。

output_tokens from 32 to 256
observe:
 decode queue grows
 TPOT and goodput become decode-limited

10. 张量级别流程图

把一次请求拆成张量和系统事件:

Request prompt:
 token_ids: [L_prompt]

Embedding:
 hidden: [1, L_prompt, H]

Prefill through transformer layers:
 layer input hidden: [1, L_prompt, H]
 Q: [1, L_prompt, n_heads, head_dim]
 K: [1, L_prompt, kv_heads, head_dim]
 V: [1, L_prompt, kv_heads, head_dim]
 layer output hidden: [1, L_prompt, H]

Saved KV cache:
 for each layer:


```
K_cache: [1, L_prompt, kv_heads, head_dim]
V_cache: [1, L_prompt, kv_heads, head_dim]
```

Transfer:

```
prefill instance sends K_cache and V_cache
decode instance receives them before continuing generation
```

Decode step t:

```
new token hidden: [1, 1, H]
read past KV: [1, L_prompt + t, kv_heads, head_dim]
output next logits: [1, vocab_size]
```

DistServe 的系统创新并没有改变 Transformer attention 的数学。它改变的是这些张量在哪些 GPU 上产生, 何时传输, 由哪个 scheduler 控制, 以及如何为不同阶段分配资源。

11. 用 AI agent 正确学习这篇论文

你的目标不是让 agent 替你总结, 而是让 agent 当教练, 帮你把论文变成自己的工作记忆。

推荐流程:

1. 你先独立读本 guide 的 0 到 6 节, 不要问 agent。
2. 合上 guide, 手画两张图: colocated 架构和 DistServe 架构。
3. 打开 distserve_original_minimal.py, 找到 goodput_at_rate 和 search_gpu_split。
4. 让 agent 只问问题, 不直接给答案。
5. 每回答一个问题, 都要用代码文件里的一个函数来对应。

可以这样提示 agent:

我正在学习 DistServe。请你扮演论文助教, 不要直接总结。

请一次只问一个问题, 按以下顺序考我:

1. prefill 和 decode 为什么会互相干扰
2. TTFT, TPOT, attainment, goodput 的区别
3. 为什么 chunked prefill 没有彻底解决问题
4. KV cache transfer 为什么既是成本也是 placement 约束
5. Algorithm 1 和 Algorithm 2 分别适合什么集群
6. 论文实验里的三种应用为什么代表不同 SLO 形态
7. 本仓库 toy simulator 的哪个函数对应论文的 goodput search

每次我回答后, 请指出一个具体漏洞, 并要求我回到代码或 guide 的某一节修正。

最后让我用 200 字复述 DistServe 的贡献。

你也可以让 agent 做反向考试:

请你给我一个 workload:

prompt length, output length, TTFT SLO, TPOT SLO, bandwidth.

让我判断 DistServe search 更可能偏 prefill 还是偏 decode。

我回答后, 你再用本仓库 distserve_original_minimal.py 的变量解释。

注意: 不要让 agent 一次性生成答案。那会让你产生“我看懂了”的错觉。正确用法是短问答, 画图, 改代码, 再复述。

12. 闭卷自测

读完后你应该能回答这些问题:

- 为什么传统 colocated serving 在总吞吐看起来不错, 但在线服务 goodput 可能差。
- TTFT 和 TPOT 分别由哪个阶段主导。
- 为什么 prefill 和 decode 的 parallelism 偏好不同。
- DistServe 为什么必须传 KV cache, 传输成本怎样估算。
- Algorithm 1 和 Algorithm 2 的核心区别是什么。
- 为什么 low node-affinity cluster 要把相同 model stage colocate 到同一 node。
- 论文的主结果 7.4x 和 12.6x 分别指什么。
- 为什么 LongBench summarization 特别能体现 TPOT 问题。
- 为什么 simulator accuracy 是 placement search 的关键证据。
- 哪些场景 DistServe 可能不如 colocated 或 chunked prefill。
- 本仓库 toy simulator 里, 哪个参数会让 search 偏向 decode GPUs。
- 如果只有一张 GPU, DistServe 的设计空间为什么基本消失。

如果这些问题需要翻 guide 才能答, 说明还在“读过”阶段; 如果能画图加代码解释, 才进入“会用”阶段。

13. 一句话复述模板

可以用这个模板练习闭卷复述:

DistServe 解决的是 LLM 在线 serving 中 prefill 和 decode colocate 导致的 TTFT/TPOT 干扰问题。它把 prefill pool 和 decode pool 拆开, 让两阶段分别选择资源、parallelism 和 batching, 再用带宽感知 placement 控制 KV cache transfer 成本。

论文用 per-GPU goodput 而不是 total tokens/s 做目标, 在 ShareGPT, HumanEval, LongBench 等任务上对比 vLLM 和 DeepSpeed-MII, 证明它能在 90% 或 99% SLO attainment 下服务更多请求或支持更紧 SLO。它的局限是需要足够 GPU, 额外模型副本, 合理网络和更复杂 orchestration。

真正掌握的标志是: 你能把这段话改写成本仓库的一个实验, 而不是只把它背下来。